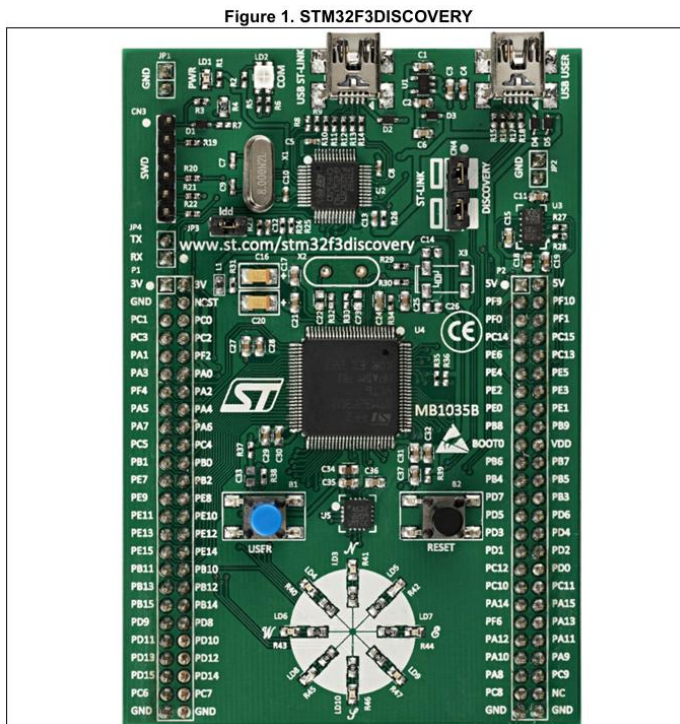


1. L'Ecosistema Hardware: SoC e Evaluation Board

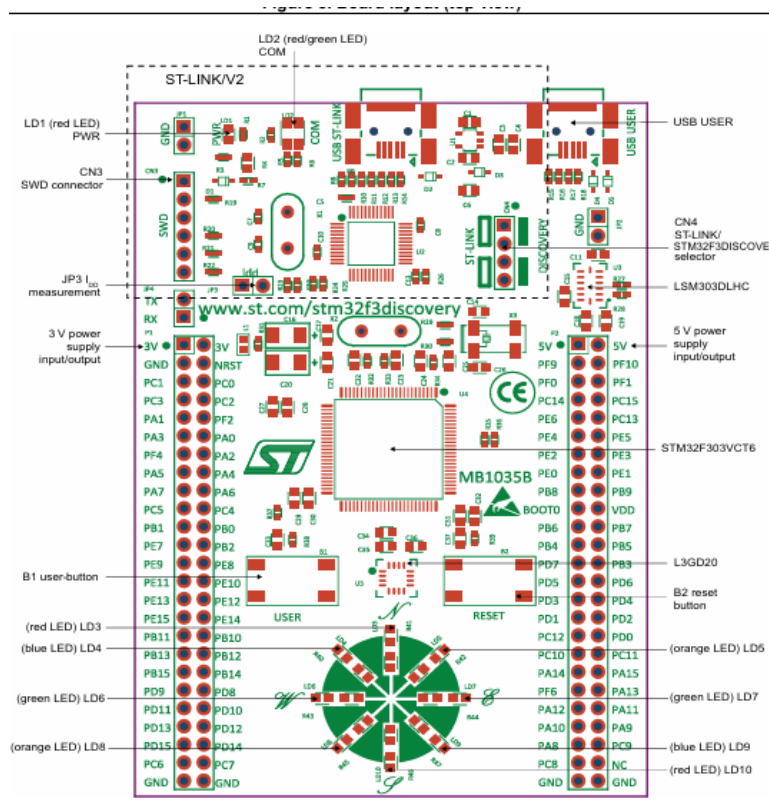
Il sistema con cui stai lavorando è diviso in due livelli principali:



- **System on Chip (SoC):** Il "cervello" centrale che contiene il microprocessore ARM, le memorie e le periferiche interne (es. Timers, controller seriali).
- **Evaluation Board:** La scheda verde che circonda il SoC. Contiene l'elettronica di contorno necessaria per programmare il chip (tramite interfaccia di test **JTAG**) e alcune periferiche utente già cablate fisicamente ai pin del SoC.

Documentazione

- 1) **User Manual:** Si focalizza sulla scheda di sviluppo specifica (es. Nucleo, Discovery). Indica a quali PIN del chip sono collegati fisicamente i LED, i pulsanti e i sensori integrati.



Nella sezione 6.4 troviamo alcune info sui LED. I LED sono tutti localizzati sui PIN della famiglia E

- LD1 PWR: Red LED indicates that the board is powered.
- LD2 COM: LD2 default status is red. LD2 turns to green to indicate that communications are in progress between the PC and the ST-LINK/V2.
- User LD3: **Red LED** is a user LED connected to the I/O PE9 of the STM32F303VCT6.
- User LD4: **Blue LED** is a user LED connected to the I/O PE8 of the STM32F303VCT6.
- User LD5: **Orange LED** is a user LED connected to the I/O PE10 of the STM32F303VCT6.
- User LD6: **Green LED** is a user LED connected to the I/O PE15 of the STM32F303VCT6.
- User LD7: **Green LED** is a user LED connected to the I/O PE11 of the STM32F303VCT6.
- User LD8: **Orange LED** is a user LED connected to the I/O PE14 of the STM32F303VCT6.
- User LD9: **Blue LED** is a user LED connected to the I/O PE12 of the STM32F303VCT6.
- User LD10: **Red LED** is a user LED connected to the I/O PE13 of the STM32F303VCT6.

Come notiamo per ogni colore ci sono due PIN differenti che possiamo accendere.

Nella sezione 6.5 troviamo alcune informazioni riguardanti I Bottini che possiamo premere:

- **B1 USER:** user and wake-up button connected to the I/O PA0 of the STM32F303VCT6.
 - **B2 RESET:** push-button connected to NRST is used to RESET the STM32F303VCT6.
- 2) **REFERENCE MANUAL (RF):** Descrive nel dettaglio l'architettura interna del microcontrollore. Contiene la mappa di memoria e la configurazione di tutti i registri delle periferiche. Descrive in dettaglio la gestione delle interruzioni
 - 3) **HAL USER MANUAL:** Descrive le librerie software (Hardware Abstraction Layer) fornite da ST

Porte I/O e Multiplexing

Il chip ha un centinaio di pin, raggruppati in porte logiche (Porta A, B, C, D, ecc.). Per risparmiare spazio fisico, i produttori utilizzano il **multiplexing**: un singolo pin fisico può svolgere diverse funzioni (es. un pin può essere un semplice ingresso digitale, oppure il pin di trasmissione di una seriale UART).

- **Scelta esclusiva:** Se configuri un pin per la comunicazione seriale, non puoi usarlo contemporaneamente come General Purpose I/O (GPIO).
 - **Pin Vincolati:** Sulla board, alcuni pin sono già collegati via hardware a componenti fisici.
-

2. Configurazione e Toolchain

L'ambiente di sviluppo (la Toolchain, tipicamente basata su interfacce grafiche come CubeMX) ti permette di configurare graficamente il multiplexing.

- **Configurazione Grafica:** Cliccando su un pin (es. PD12) e impostandolo come GPIO_Output, il pin si colora di verde nell'interfaccia, segnalando che la risorsa è allocata.
- **Generazione del Codice:** Una volta scelte periferiche e protocolli (es. seriale sincrona a 8 bit), il tool genera automaticamente la funzione di inizializzazione nel main e tutto il codice di setup.
- **Regola d'Oro del Codice Auto-Generato:** Devi scrivere il tuo codice **esclusivamente** nelle aree tratteggiate, ovvero tra i commenti `/* USER CODE BEGIN */` e `/* USER CODE END */`. Se scrivi fuori da queste aree, alla successiva rigenerazione del progetto, il tuo lavoro verrà inesorabilmente cancellato.

3. Gestione degli Input: Il Problema del Tasto

Quando si legge un input fisico come il tasto su PA0, si affronta il problema del **Rimbalzo Meccanico (Bouncing)**. Sforando il tasto, i contatti metallici "rimbalzano" per qualche millisecondo prima di stabilizzarsi. Dato che il processore esegue circa 16 milioni di cicli al secondo, interpreterà questi rimbalzi come decine di pressioni distinte.

Per leggere l'input, hai due paradigmi a disposizione. Ecco il confronto pratico:

Paradigma	Come Funziona	Vantaggi	Svantaggi
Polling	Il processore controlla continuamente lo stato del pin all'interno di un ciclo ripetitivo.	Semplice da implementare.	Spreca cicli di CPU in attesa attiva e blocca l'esecuzione di altre istruzioni.
Interrupts	Il processore fa altro. Al cambio di stato del pin, l'hardware interrompe il flusso principale e lancia una funzione specifica.	Altamente efficiente e reattivo.	Richiede la gestione del rimbalzo meccanico (tramite condensatori o logica software).

4. Architettura delle Interruzioni (NVIC e EXTI)

Per usare gli interrupt, devi abbandonare il controllo diretto dal main() e affidarti alle **ISR (Interrupt Service Routines)**.

Il SoC utilizza l'**NVIC** (Nested Vectored Interrupt Controller), un gestore integrato direttamente al processore che regola le priorità e permette l'annidamento degli interrupt.

Quando una periferica (es. un timer, l'ADC o l'USART) genera un interrupt, la CPU non deve eseguire codice software per capire chi ha chiamato. L'hardware dell'NVIC identifica la sorgente,

calcola automaticamente l'offset e va a leggere l'indirizzo della *Interrupt Service Routine* (ISR) direttamente da una **Vector Table** precaricata in memoria flash. Questo azzerà il tempo di decodifica della sorgente dell'interrupt.

Mentre le periferiche interne (come i Timer o l'ADC) sono cablate direttamente all'NVIC, i pin GPIO (General Purpose Input/Output) devono passare obbligatoriamente attraverso l'EXTI.

L'EXTI gestisce 16 linee principali per gli interrupt esterni, chiamate da **EXTI0** a **EXTI15**. Il numero della linea corrisponde *esattamente* al numero del pin. Questo crea un vincolo architetturale fondamentale: tutti i Pin 0 (PA0, PB0, PC0...) convergono sulla linea EXTI0 attraverso un multiplexer (chiamato SYSCFG). Non puoi avere un interrupt simultaneo su PA0 e PB0.

L'STM32 non spreca 16 vettori NVIC separati per le 16 linee EXTI. Fa un raggruppamento per risparmiare risorse:

- **EXTI0, EXTI1, EXTI2, EXTI3, EXTI4:** Hanno un vettore NVIC e una routine (ISR) dedicata per ciascuno.
- **EXTI5 fino a EXTI9:** Condividono un unico vettore NVIC (`EXTI9_5_IRQn`).
- **EXTI10 fino a EXTI15:** Condividono un unico vettore NVIC (`EXTI15_10_IRQn`).

Cosa significa in pratica? Se usi il vettore `EXTI9_5`, quando scatta l'interrupt la CPU entra nella tua ISR, ma non sa *quale* pin lo ha generato. Devi essere tu via software a leggere il registro di stato dell'EXTI per capire se è stato il pin 7, l'8 o il 9.

Le interruzioni esterne provenienti dai pin passano per il controller **EXTI** (External Interrupt/Event Controller). Se vogliamo usare il tasto User come interruzione per accendere un LED dobbiamo prima capire dove si trova tramite USER MANUAL. Si trova sul PIN PA0 di GPIO. Ora bisogna vedere su che linea di interruzione è collegato. Andiamo nel reference manual e troviamo i registri delle interruzioni

12.1.3 31 SYSCFG external interrupt configuration register 1 ad esempio questo

E bisogna trovare il PIN che ci interessa.

Bits 3:0 EXTI0[3:0]: EXTI 0 configuration bits These bits are written by software to select the source input for the EXTI0 external interrupt.

x000: PA[0] → trovato quindi è collegato su EXTI0.

pin x001: PB[0]

pin x010: PC[0]

pin x011: PD[0]

pin x100: PE[0]

pin x101: PF[0]

pin x110:PG[0]

pin x111:PH[0]

pin Note: other configurations: reserved

In un progetto STM32CubeIDE, il flusso di gestione delle interruzioni e le relative callback sono tracciabili attraverso tre file principali:

- **startup_stm32f407vgtx.s**: in cui viene inizializzata la Vector Table ed è possibile identificare i nomi degli Handler delle interruzioni.
- **stm32f4xx_it.c**: in cui ci sono le effettive implementazioni degli Handler delle interruzioni.
- **stm32f4xx_hal_*nome_periferica*.c**: in cui vengono trovate le funzioni weak da dover copiare nel main e modificare.

Quando l'EXTI rileva un fronte valido, alza un flag hardware a '1' nel registro `PR` (Pending Register) e avvisa l'NVIC.

Attenzione: A differenza di altri microcontrollori, l'hardware dell'STM32 *non* azzerava questo flag automaticamente quando entri nell'ISR. Se esci dalla routine senza azzerarlo manualmente (scrivendo uno '1' nel bit corrispondente), l'NVIC penserà che l'interrupt sia ancora attivo e richiamerà l'ISR all'infinito.

Usando le librerie HAL, non hai bisogno di toccare le `IRQHandler` menzionate sopra. Il codice generato da ST dentro quelle routine si occupa già di fare il "lavoro sporco": legge il Pending Register (PR) per capire quale pin ha generato l'interrupt, **azzerava automaticamente il flag hardware** per evitare loop infiniti, e infine chiama una singola funzione unificata:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin).
```

E' definita con l'attributo `__weak` (debole). Questo significa che possiamo "sovrascriverla" semplicemente ridefinendola nel `main.c`.

L'argomento `GPIO_Pin` che la funzione riceve in ingresso è la chiave per risolvere il problema dei "vettori condivisi". Ti dice esattamente quale pin ha superato i filtri ed è arrivato all'NVIC

Il Flusso Pratico dell'Interrupt

1. **Configurazione:** Imposti graficamente il pin PA0 su `EXTI_Line0` e abiliti l'interrupt corrispondente nell'NVIC.
2. **Vector Table:** Le interruzioni sono vettorizzate. In memoria esiste una tabella contenente l'indirizzo di memoria della funzione ISR da eseguire per ogni specifico evento.
3. **Entry Point e Handler:** Quando l'hardware rileva la pressione, va nel file di startup e chiama l'handler base (es. `EXTI0_IRQHandler`).
4. **La Callback Utente:** L'handler a sua volta chiama automaticamente una funzione di **Callback**. La funzione che stai cercando è `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`. Questa funzione è pre-dichiarata in modo debole (weak) dal sistema. Per usarla, devi "sovrascriverla" nel tuo codice in modo statico: scrivi l'istituzione della

funzione nel tuo file sorgente e ci metti dentro la logica per accendere e spegnere il LED.

5. Astrazione: Dal Modello ASIM all'HAL

Rispetto ad architetture o modelli didattici (come ASIM), in questo contesto ti muovi su un livello di astrazione superiore.

- **In ASIM:** Manipoli i registri nudi e crudi, prendendo una periferica parallela e definendone manualmente il comportamento.
- **In HAL (Hardware Abstraction Layer):** Usi API pronte. Il livello HAL fornisce funzioni già testate dal produttore (come le System Call) che ti nascondono la complessità hardware.

Tuttavia, hai a disposizione una doppia opportunità. Se usi un protocollo standard (come la Seriale), il codice generato gestisce quasi tutto. Se invece devi comunicare con un sensore che usa un protocollo parallelo non standard, potresti dover comandare singolarmente 10 fili alzandoli e abbassandoli via codice (bit-banging).

6. Il Mondo dei Timer (Integrazione Aggiuntiva)

Nei tuoi appunti hai menzionato il timer e la stringa `HAL_TIM_Base_Stop_IT`. I timer funzionano in modo analogo all'EXTI, ma legati allo scorrere del tempo.

- **Avvio:** Per iniziare a contare (e non fermare), dovrai lanciare la funzione `HAL_TIM_Base_Start_IT(&htim6)` (dove IT sta proprio per Interrupt).
- **L'Evento:** Quando il timer finisce di contare il periodo stabilito, genera un'interruzione.
- **La Callback:** Esattamente come per i pin fisici, la libreria chiamerà automaticamente una funzione. In questo caso è `HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)`. Al suo interno scriverai il codice da eseguire allo scadere del tempo. Questo è, tra l'altro, il metodo più elegante per risolvere via software il "problema meccanico" dei rimbalzi del tasto.

Primo progetto: accendiamo Luce in seguito al tasto USER premuto.

Creiamo un nuovo progetto e nel file generato con estensione `.ioc` ci comparirà graficamente la board. Troviamo il pin PA0 (tasto USER) e impostiamolo su GPIOEXTIO per metterlo come interruzione.

E il PIN PE10 (led arancione) su GPIO output.

Andiamo ora a sinistra su system core e cerchiamo NVIC. Cerchiamo poi EXTI interrupt line 0 e assicuriamoci che la casella sia spuntata.

Per confermare e generare il codice salviamo con `ctrl+s`

Andiamo nel file startup per trovare l'handler di EXTI0. Che corrisponde a `EXTI0_IRQHANDLER`

Ora dobbiamo trovare l'implementazione di IRQhandler. Andiamo nel file stm32f4xx_it.c e lo troviamo.

All'interno troviamo il nome dell'effettiva implementazione ovvero

HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);

e quindi ora dobbiamo andare nel file

stm32f4xx_hal_gpio.c che si trova nella cartella driver

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    /* EXTI line interrupt detected */
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
    {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}

/**
 * @brief EXTI line detection callback.
 * @param GPIO_Pin Specifies the port pin connected to corresponding EXTI line.
 * @retval None
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
```

Ci interessa la funzione callback. Dobbiamo ridefinirla nel codice.

Possiamo scrivere dove dice user code begin e ridefiniamo quella funzione

/* USER CODE BEGIN 4 */

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    // 1. Verifichiamo che l'interruzione provenga dal Tasto User (PA0)
    if (GPIO_Pin == GPIO_PIN_0)
    {
        // 2. Leggiamo lo stato attuale del LED sulla porta E, pin 9
        GPIO_PinState stato_led = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_9);

        // 3. Applichiamo la logica condizionale
        if (stato_led == GPIO_PIN_RESET)
        {
            // Se il LED è spento, lo accendiamo
            HAL_GPIO_WritePin(GPIOE, GPIO_PIN_9, GPIO_PIN_SET);
        }
        else
        {
            // Se il LED è già acceso, lo spegniamo
            HAL_GPIO_WritePin(GPIOE, GPIO_PIN_9, GPIO_PIN_RESET);
        }
    }
}
```

Quindi abbiamo imparato due funzioni:

- HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_9) per leggere lo stato del pin
- HAL_GPIO_WritePin(GPIOE, GPIO_PIN_9, GPIO_PIN_SET) per modificarlo. SET lo accende mentre RESET lo spegne.

